MedConnect: Clinic Appointment Management System

Final Database Design & Application Submission

Group-11 | P2 Section | November 2026

2022A7PS0059H	Charan Teja Valavala
2023A3PS0304H	Rithwik Sai Madadi
2023A3PS0325H	Peketi Sai Manoj
2023A3PS0367H	Y Sameeksha
2023A3PS0368H	K S Rajeev Kasyap

1. Problem Statement, Scope & Application Design

Objective. Integrate patient registration, provider schedules, appointment booking, clinical encounters, pharmacy dispensing, billing, and reporting for a multi-specialty outpatient clinic. The database remains the core: the application exercises ten related tables through parameterized SQL and transactional workflows.

Intended Users & Assumptions

- **Admin/Receptionist:** maintains patient entries, books/reschedules/cancels visits, manages the medicine catalog and stock receipts, records payments, and reads clinic reports.
- **Doctor:** views assigned appointments and relevant patient records, starts consultations, and completes clinical notes and prescriptions. Patients are data entities; there is no patient login.
- One doctor belongs to one department. Shifts recur weekly, within one day; the clinic uses Asia/Kolkata time and INR. Appointment duration is captured from its shift.
- A visit has at most one medical record, prescription header, and invoice. A prescription can contain multiple distinct medicines. Historical records may have a null appointment link.
- Cancelled bookings remain in history and release their slots. Completed notes, dispenses, and issued invoice amounts cannot be edited through the supported workflows.

Integrated End-to-End Workflows

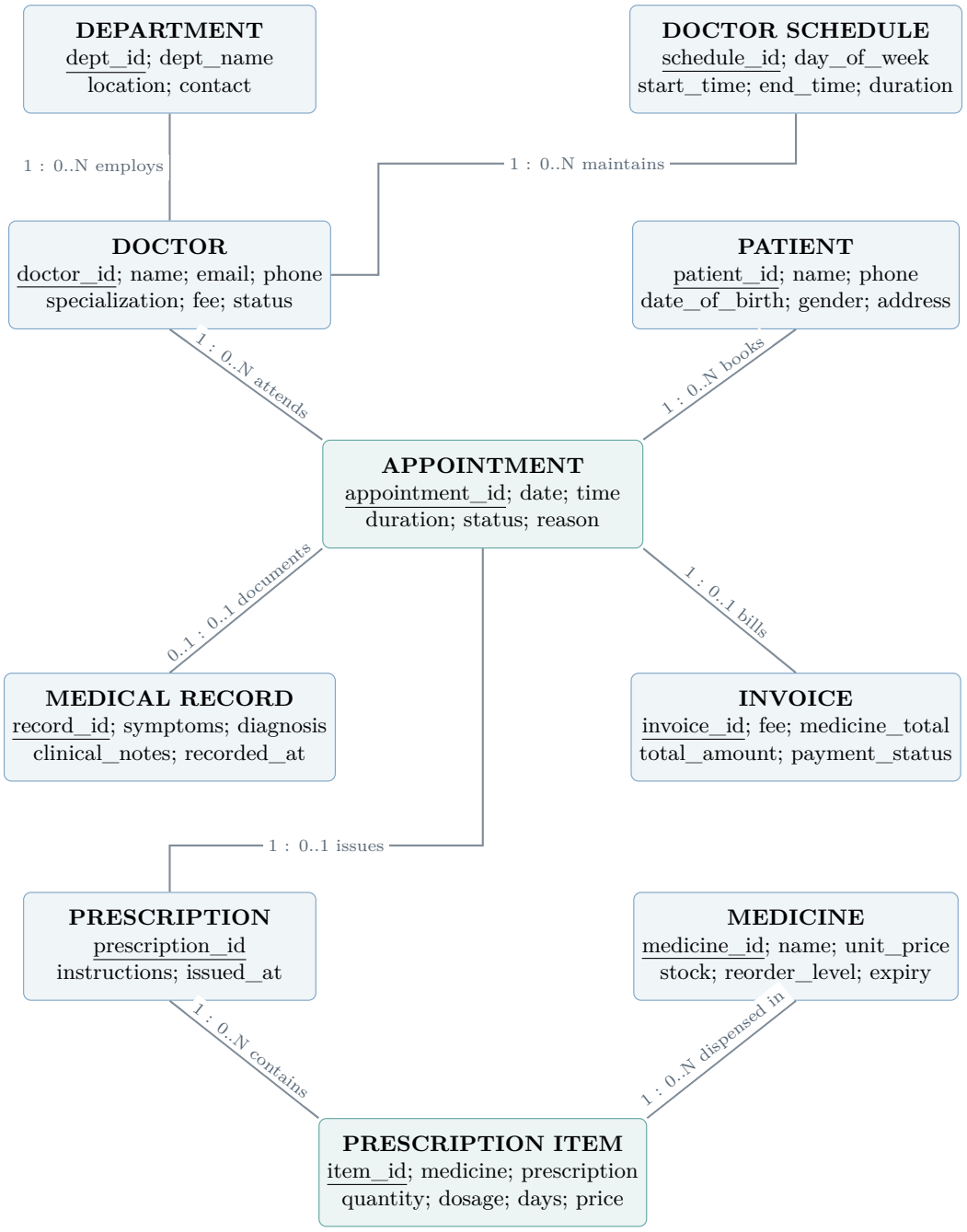
1. **Booking & concurrency:** reception registers/selects a patient, retrieves an active doctor's available times, and books a future visit. Shift checks, a provider row lock, overlap detection, and a partial unique index reject collisions. Cancellation permits rebooking.
2. **Clinical encounter & pharmacy:** the assigned doctor starts a visit, records findings, and submits prescription lines. A trigger locks each medicine, rejects expired/insufficient stock, captures the unit price, and deducts quantity. A failure rolls back the entire encounter.
3. **Billing & analytics:** encounter completion calls the invoice procedure. Reception records a payment; the paid-only department view and SQL reports reflect the change.

Technology Stack & Architecture

React 18 + Vite + Tailwind CSS + TanStack Query
Staff/doctor interface, forms, schedule, inventory, reports
↕ REST/JSON with bearer authentication
FastAPI + Pydantic + asyncpg ⟷ PostgreSQL 16
Raw bound SQL, connection pool, transactions, PL/pgSQL routines

2. Final Conceptual ER Model

Entities have independent surrogate identifiers. Dependent existence alone does not make an entity weak in the identifying-key sense. Prescription–Medicine is an M:N relationship resolved by `Prescription_Items`. The diagram shows minimum/maximum participation for the primary links; direct provenance links are listed below and on page 3.



Provenance links. Each record and prescription references exactly one patient and doctor; each invoice references one patient. Linked identities must match the appointment. Records may exist without an appointment for imported history. A header can have zero lines at the relational level; the application only creates a prescription when medicines are selected.

3. Relational Schema, Keys & Domains

Notation: underlined attributes are primary keys; ∗ marks foreign keys; ? marks nullable attributes. `created_at` is a timezone-aware timestamp where listed. All identifier keys are auto-incrementing integers. Full executable DDL is in `schema.sql`.

1. Departments

(dept_id, dept_name, description?, location, contact_number?, created_at)

2. Doctors

(doctor_id, dept_id*, first_name, last_name, email, phone, specialization, qualification, consultation_fee, status, password_hash?, created_at)

3. Doctor_Schedules

(schedule_id, doctor_id*, day_of_week, start_time, end_time, slot_duration_minutes, is_active)

4. Patients

(patient_id, first_name, last_name, date_of_birth, gender, blood_group?, phone, email?, address, emergency_contact_phone?, created_at)

5. Appointments

(appointment_id, patient_id*, doctor_id*, appointment_date, appointment_time, duration_minutes, status, reason_for_visit?, created_at)

6. Medical_Records

(record_id, appointment_id*?, patient_id*, doctor_id*, symptoms, diagnosis, clinical_notes?, recorded_at)

7. Prescriptions

(prescription_id, appointment_id*, patient_id*, doctor_id*, instructions?, issued_at)

8. Medicines

(medicine_id, name, category, manufacturer, unit_price, stock_quantity, reorder_level, expiry_date, created_at)

9. Prescription_Items

(item_id, prescription_id*, medicine_id*, dosage, duration_days, quantity, unit_price)

10. Invoices

(invoice_id, appointment_id*, patient_id*, consultation_fee, medicine_total, total_amount, payment_status, payment_method?, issued_at, paid_at?)

Foreign Keys & Referential Actions

Child relation	Referenced parent(s)	On delete
Doctors	Departments	RESTRICT
Doctor_Schedules	Doctors	CASCADE
Appointments	Patients; Doctors	RESTRICT
Medical_Records	Appointments; Patients; Doctors	SET NULL; RESTRICT
Prescriptions	Appointments; Patients; Doctors	RESTRICT
Prescription_Items	Prescriptions; Medicines	CASCADE; RESTRICT
Invoices	Appointments; Patients	RESTRICT

The DDL retains these referential actions from the mid-sem model; final retention triggers additionally block deleting appointments, schedules, clinical records, prescriptions, dispenses, and invoices. Cascades therefore do not provide a supported way to erase clinical history.

Candidate Keys & Attribute Domains

Unique: department name; doctor email and phone; patient phone and non-null email (within their respective tables); medicine name; (doctor, weekday, shift start); (prescription, medicine); non-null appointment in records; appointment in prescriptions and invoices. Money uses `NUMERIC(10,2)`; stock/quantity are integers; visit dates use `DATE`; local slots use `TIME`; event timestamps use `TIMESTAMPTZ`. Names/contact fields use bounded strings and clinical notes use `TEXT`.

4. Integrity Constraints, Transactions & Normalization

Database-Enforced Rules

Rule	Enforcement and outcome
Identity and existence	Ten primary keys and foreign keys preserve references. Clinical-link triggers reject patient/doctor mismatches; invoice identity must match the completed visit.
Domains and arithmetic	CHECK validates allowed statuses, positive quantities/days, nonnegative prices/stock/fees, ordered shift times, and invoice total = consultation fee + medicine total.
Booking availability	Active provider; matching weekday shift; exact slot alignment; duration snapshot; provider row lock; overlap check. The partial unique index excludes only Cancelled rows.
Lifecycle and retention	Scheduled → In_Consultation → Completed, or Scheduled → Cancelled/No_Show. A clinical record is required for completion. Only Scheduled visits can move. Historical/clinical deletes and edits are rejected.
Pharmacy	BEFORE INSERT trigger locks the medicine, checks expiry/stock, snapshots its price, and deducts quantity. Duplicate medicine lines are unique per prescription. Dispenses are retained.
Billing and payment	Procedure locks the appointment, requires Completed, sums dispensed price snapshots, and returns on a repeated call. Insert trigger checks fee/medicine amounts. Issued amounts are fixed; supported payment transition is Pending → Paid.

Transaction Boundaries & Concurrency

The backend uses `asyncpg` transactions at PostgreSQL's default READ COMMITTED isolation. Completion writes the medical record, prescription and lines, stock changes, status, and invoice in one transaction. Any invalid medicine aborts all of them. Item locks are acquired in medicine-ID order. Invoice generation and payment serialize on their parent rows; repeating payment with the same method returns the existing result.

The availability list is advisory: a competing booking can win before confirmation. The database rechecks the write; the API returns HTTP 409 and the interface offers refreshed times. Row locks preserve stock under competing prescriptions, rather than relying on the frontend's displayed quantity.

Normalization & Deliberate Redundancy

- **1NF:** atomic attributes, with medication lines in a separate relation. The history report builds JSON at query time; it is not stored as repeating groups.
- **2NF/3NF:** catalog, department, patient, provider, and schedule facts are separated. Prescription-item attributes depend on the whole (prescription, medicine) key. Department descriptions are not repeated in doctor rows.
- **BCNF under the stated dependencies:** master/schedule/appointment/item relations use superkey determinants. A nullable UNIQUE appointment link in records is not an unconditional relational candidate key; multiple legacy rows may have null links.
- **Controlled redundancy:** linked clinical headers repeat patient/doctor IDs for provenance, enforced by triggers. Invoice patient is derived from its unique appointment. `total_amount` is stored for billing, despite the non-key dependency (consultation fee, medicine total) → total. Thus a universal BCNF claim is not made. A CHECK maintains this arithmetic.
- **Historical prices:** dispensed unit price and issued consultation fee are time-specific facts, independent of later catalog changes. They prevent retroactive repricing.

Temporal validation. Stable domain checks remain in DDL. The API validates future birth dates and past bookings; dispensing expiry is checked at write time. PostgreSQL assumes CHECK conditions remain valid for the same row values, so cross-row and time-sensitive workflow rules belong in triggers/application validation.

5. SQL Evidence, Final Decisions & Team Responsibilities

Five Executable Reports & Additional DBMS Features

SQL report in database/queries	Purpose and concepts exercised
01_department_revenue.sql	Current-month billed, collected, and pending amounts by specialty; joins, CTE, filtered aggregates, DENSE_RANK, and SUM OVER.
02_doctor_performance.sql	Last-30-day completed/cancelled/no-show counts, attendance and collections; outer joins, grouping, FILTER and NULLIF.
03_pharmacy_reorder.sql	Low/expired stock, recent dispensing velocity and estimated days remaining; two CTEs, joins, CASE and guarded division.
04_patient_history.sql	Patient encounters and dispensed medication snapshots; multi-table joins, correlated subquery, ordered JSON aggregation; bound patient/doctor parameters.
05_peak_demand.sql	Last-90-day visits by weekday/hour and no-show rate; date extraction, grouped CTE, filtered counts and RANK.

Additional features: explicit transactions and row locks; PL/pgSQL triggers; stored invoice procedure; paid-only department revenue view; partial unique and query-supporting B-tree indexes. Their use follows the three workflows rather than adding unrelated objects.

Important Final Design Decisions

1. Retain the ten-table mid-sem scope and raw SQL stack. Add doctor password hashes, appointment duration, and dispensed unit-price snapshots without an extra account table. The administration identity is configured outside the database.
2. Treat doctors and administration as distinct application roles. Signed, expiring tokens and PBKDF2 password hashes protect staff access; doctor reads are scoped to assigned encounters. No patient self-service, insurance adjudication, refunds, split payments, or multi-clinic operation is implemented.
3. Preserve clinical and billing history. Corrections/refunds would need a separate, designed amendment workflow; issued records are not silently overwritten.
4. Keep reproducibility explicit: schema, synthetic seed script, environment generator, dependency locks, Docker Compose, native instructions, and PostgreSQL-backed tests. Lists are bounded for the lab demonstration; the reports query the underlying relations directly.

Group-11: Continued Roles & Final Module Responsibilities

Member	BITS ID	Responsibility
K S Rajeev Kasyap	2023A3PS0368H	ER/relational diagrams, schema, keys, indexes, schedule and booking design.
Rithwik Sai Madadi	2023A3PS0304H	Backend/database integration, patient and booking APIs, staff access, setup.
Peketi Sai Manoj	2023A3PS0325H	Clinical integrity, stock trigger, invoice procedure, concurrency, normalization.
Y Sameeksha	2023A3PS0367H	React interface, staff/doctor workflows, validation, responsive layout, demo.
Charan Teja Valavala	2022A7PS0059H	SQL reports, sample-data/verification coordination, final report, README, viva preparation.

These responsibilities continue the submitted mid-sem roles. Every member must understand the entire relational design and reproduce the integrated workflows for the individual viva.

Reproduction & Verification Route

Run `scripts/setup_env.py`, then follow the native or Compose route in README. A fresh seed creates **1,308 synthetic records** across ten relations. `scripts/verify_db.py` runs all five reports. PostgreSQL tests cover keys/domains, slot races, stock races, rollback, pricing, invoice retries, permissions, and HTTP workflows. `docs/demo_guide.md` gives the live-demo sequence; `docs/verification.md` records verified results and limits.

References: DBMS Project Guidelines, Sections 3–8; PostgreSQL 16 documentation, Sections 5.4 (constraints), 11.8 (partial indexes), and 13.3 (explicit locking). Repository: `https://github.com/charanvalavala/medconnect-final-submission`.